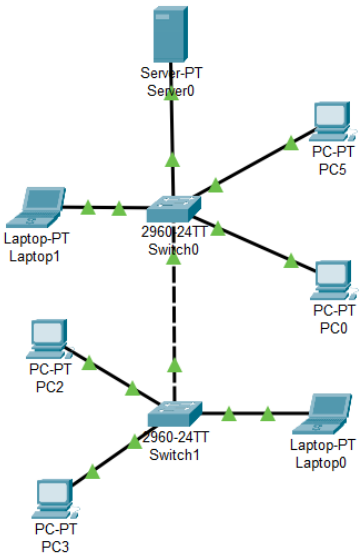
Name: Sri Ragavendra S

- Simulate a small network with switches and multiple devices. Use ping to generate traffic and observe the MAC address table of the switch. Capture packets using Wireshark to analyze Ethernet frames and MAC addressing.
- Capture and analyze Ethernet frames using Wireshark. Inspect the structure of the frame, including destination and source MAC addresses, Ethertype, payload, and FCS. Use GNS3 or Packet Tracer to simulate network traffic.
- Configure static IP addresses, modify MAC addresses, and verify network connectivity using ping and ifconfig commands.
- Troubleshoot Ethernet Communication with ping and traceroute -> Using cisco packet tracer:
- Create a simple LAN setup with two Linux machines connected via a switch.
- Ping from one machine to the other. If it fails, use ifconfig to ensure the IP addresses are configured correctly.
- Use traceroute to identify where the packets are being dropped if the ping fails.

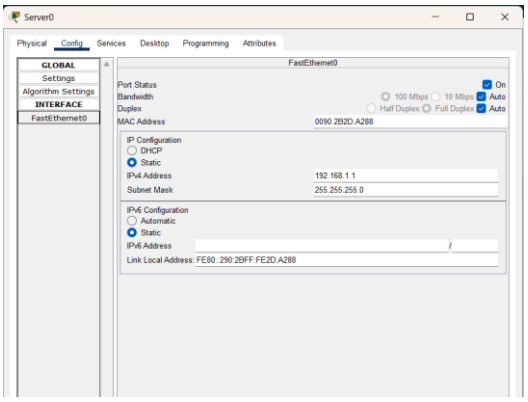
<u>The network:</u>



The connections are perfect, and the link is closed as there is green arrow marks in between the connections.

We need to set the ip address on our own,

Example:



Similarly, we need to set it for all other devices.

<u>The IP address:</u>

Device	IP	Mask
Server0	192.168.1.1	255.255.255.0
Laptop1	192.168.1.10	255.255.255.0
PC5	192.168.1.20	255.255.255.0
PC0	192.168.1.30	255.255.255.0
Laptop0	192.168.1.40	255.255.255.0
PC2	192.168.1.50	255.255.255.0
PC3	192.168.1.60	255.255.255.0

<u>Confirmation of the connection of two devices using the switch:</u>

I have used the ping command:

```
C:\>ping 192.168.1.20

Pinging 192.168.1.20 with 32 bytes of data:

Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time<1ms TTL=128
Reply from 192.168.1.20: bytes=32 time=1ms TTL=128

Ping statistics for 192.168.1.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>
```

I got the reply, this means the ping is working as expected and the connection is correct

After generating traffic using the ping command, the switch dynamically learned the MAC addresses of connected devices. The command show mac address-table was used to display the MAC address table. The table shows the MAC addresses and the switch ports through which the devices are reachable.

Column	Meaning
VLAN	The VLAN associated with the MAC address
MAC Address	Hardware address of the device
Type	Whether the entry is learned dynamically
Port	Switch port where the device is connected

Switch 0:

```
Switch#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0002.1696.1a64    DYNAMIC     Fa0/2
   1    000c.cf2a.3e01    DYNAMIC     Fa0/3
   1    0030.a30d.132e    DYNAMIC     Fa0/4
Switch#
```

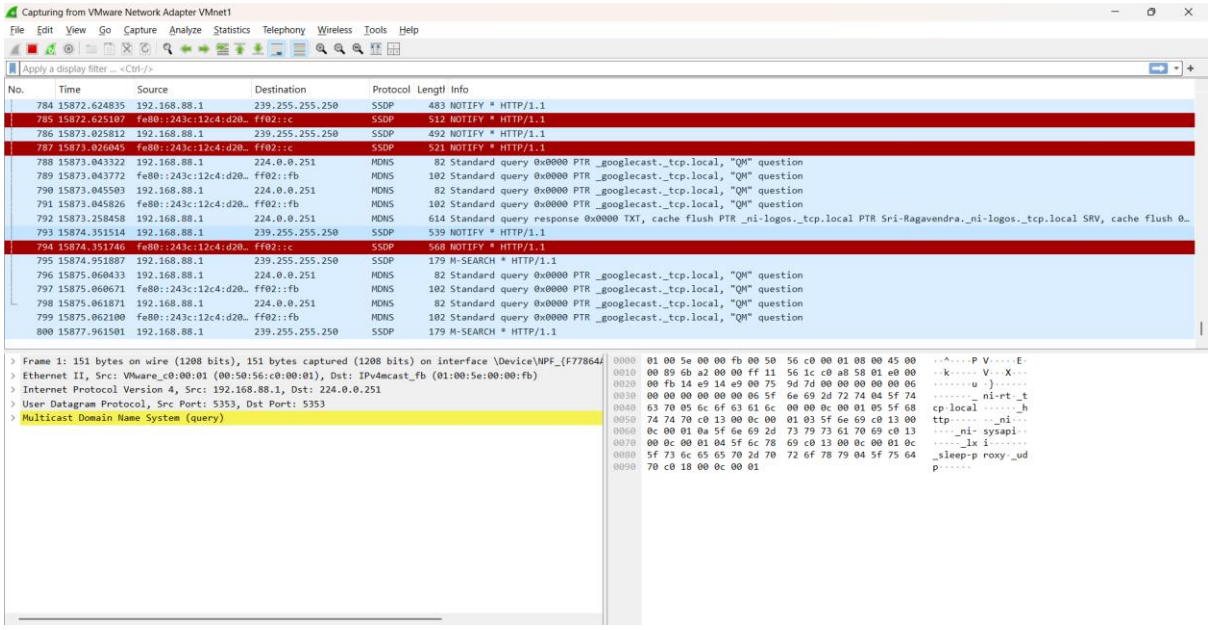
Switch 1

```
Switch>enable
Switch#show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0002.16a6.b003    DYNAMIC     Fa0/1
```

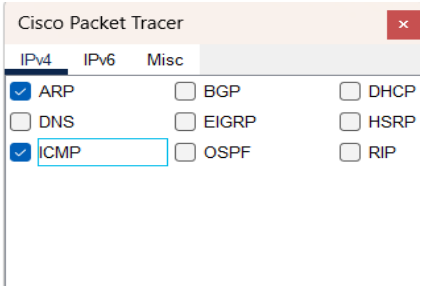
The **Type field shows DYNAMIC**, meaning the switch automatically learned these MAC addresses from incoming Ethernet frame

Column	Meaning
VLAN	The VLAN associated with the MAC address
MAC Address	Hardware address of the device
Type	Whether the entry is learned dynamically
Port	Switch port where the device is connected

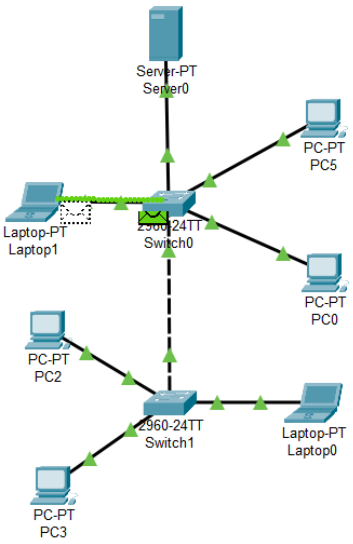
<u>The packets observed using WireShark:</u>



Now we can observe the packet flow (ARP + ICMP)



The packet flow:



Packet Tracer simulation mode was used to observe the packet exchange between devices. When a device initiates communication using the ping command, the Address Resolution Protocol (ARP) is first used to determine the MAC address of the destination device. The source device sends an ARP request, which is broadcast across the network. The destination device replies with an ARP response containing its MAC address. After resolving the MAC address, ICMP packets are exchanged to complete the ping operation.

```
C:\>ping 192.168.1.20

Pinging 192.168.1.20 with 32 bytes of data:

Reply from 192.168.1.20: bytes=32 time=4ms TTL=128
Reply from 192.168.1.20: bytes=32 time=4ms TTL=128
Reply from 192.168.1.20: bytes=32 time=4ms TTL=128
Reply from 192.168.1.20: bytes=32 time=4ms TTL=128

Ping statistics for 192.168.1.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 4ms, Maximum = 4ms, Average = 4ms
```

To verify network connectivity, the **ping command** was executed from one device to another device within the same LAN.
The command ping 192.168.1.20 was used to test communication between the devices.

The output shows that **4 packets were sent and 4 packets were received with 0% packet loss**, indicating successful communication between the devices.
The round-trip time was approximately **4 ms**, which confirms that the network is functioning properly.

The TTL (Time To Live) value of **128** indicates that the packet was successfully delivered without passing through multiple routing devices, which is expected in a simple LAN environment.

<u>This is the flow:</u>

Laptop1 → Switch0 → Broadcast ARP

PC5 → ARP Reply → Laptop1

Laptop1 → ICMP Echo Request

PC5 → ICMP Echo Reply

Using traceroute to identify where the packets are being dropped if the ping fails

```
hanuman@hanuman-VMware-Virtual-Platform:~$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  _gateway (192.168.23.2)  1.323 ms  1.132 ms  1.049 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
```

- The system is tracing the route to **Google DNS (8.8.8.8)**.
- It will try a maximum of **30 hops** (routers).
- Each probe packet is **60 bytes**.

Research the Linux kernel's handling of Ethernet devices and network interfaces. Write a short report on how the Linux kernel supports Ethernet communication (referencing kernel.org documentation).

The Linux kernel manages Ethernet communication through its networking subsystem, which includes network device drivers, the TCP/IP protocol stack, and network interfaces. Each Ethernet device is represented in the system as a network interface such as **eth0, ens33, or wlan0**. The kernel communicates with hardware through **device drivers**, which allow the operating system to send and receive Ethernet frames.

When a packet is transmitted, the data passes through the **TCP/IP stack** in the kernel, where it is encapsulated into appropriate protocol headers such as IP and Ethernet. The kernel then forwards the frame to the network interface driver, which sends it to the physical Ethernet device. Similarly, when an Ethernet frame is received from the network, the driver passes it to the kernel, which processes the frame and delivers the data to the correct application through sockets.

The Linux kernel also maintains structures such as the **ARP table**, routing table, and interface configuration to manage communication between devices. Tools such as ip, ifconfig, and ip neigh allow users to inspect and configure network interfaces managed by the kernel.

The Linux networking subsystem therefore plays a critical role in enabling reliable Ethernet communication by managing device drivers, packet processing, and interface configuration.

Reference: Linux Kernel Networking Documentation – https://www.kernel.org/doc/

Describe how you would configure a basic LAN interface using the ip command in Linux (kernel.org).

In Linux, network interfaces can be configured using the ip command, which is part of the iproute2 package. This command allows administrators to view, configure, and manage network interfaces and IP addresses.

```
hanuman@hanuman-VMware-Virtual-Platform:~$ sudo ip addr add 192.168.1.10/24 dev eth0
[sudo] password for hanuman:
Cannot find device "eth0"
hanuman@hanuman-VMware-Virtual-Platform:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:30:d5:21 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
hanuman@hanuman-VMware-Virtual-Platform:~$ sudo ip addr add 192.168.1.10/24 dev ens33
hanuman@hanuman-VMware-Virtual-Platform:~$ sudo ip link set ens33 up
hanuman@hanuman-VMware-Virtual-Platform:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:30:d5:21 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.23.129/24 brd 192.168.23.255 scope global dynamic noprefixroute ens33
       valid_lft 1746sec preferred_lft 1746sec
    inet 192.168.1.10/24 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe30:d521/64 scope link
       valid_lft forever preferred_lft forever
```

Use Linux to view the MAC address table of a switch (if using a Linux-based network switch). Use the bridge or ip link commands to inspect the MAC table and demonstrate a basic switch's operation.

```
hanuman@hanuman-VMware-Virtual-Platform:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
 group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mod
e DEFAULT group default qlen 1000
    link/ether 00:0c:29:30:d5:21 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
hanuman@hanuman-VMware-Virtual-Platform:~$
```

Field	Meaning
ens33	Network interface
link/ether	MAC address
brd	Broadcast address

```
hanuman@hanuman-VMware-Virtual-Platform:~$ ip neigh
192.168.23.2 dev ens33 lladdr 00:50:56:fd:ca:40 REACHABLE
192.168.23.254 dev ens33 lladdr 00:50:56:f3:41:6e STALE
192.168.23.1 dev ens33 lladdr 00:50:56:c0:00:08 STALE
hanuman@hanuman-VMware-Virtual-Platform:~$
```

Field	Meaning
192.168.23.2	IP address
lladdr	MAC address
REACHABLE	Connection status

Linux networking commands were used to inspect MAC addresses and ARP mappings. The command ip link show displays the available network interfaces along with their MAC addresses. In this system the active interface is ens33, and the link/ether field shows the MAC address of the device.

The command ip neigh displays the ARP table, which shows the mapping between IP addresses and MAC addresses of devices in the local network. The lladdr field represents the MAC address of the neighboring device, while the status field (such as REACHABLE or STALE) indicates the state of the

connection. These commands help demonstrate how Linux learns and stores hardware addresses for communication within a LAN.

In this assignment, I created a small LAN network using Cisco Packet Tracer with switches and multiple devices. I configured static IP addresses for the devices and tested the connectivity between them using the ping command. The successful ping replies confirmed that the devices were able to communicate properly through the switch.

I also checked the MAC address table of the switch to see how the switch learns the MAC addresses of connected devices dynamically. Using Packet Tracer simulation mode, I observed the packet flow such as ARP request, ARP reply, and ICMP packets during communication between devices.

Wireshark was used to capture and analyze Ethernet frames. From the captured packets, I examined the Ethernet frame fields including the source MAC address, destination MAC address, EtherType, and payload.

In Linux, I used commands such as ip link show and ip neigh to view network interfaces, MAC addresses, and ARP mappings. I also used ping and traceroute commands to test and troubleshoot network communication.

Overall, this assignment helped me understand how devices communicate in a LAN, how switches learn MAC addresses, and how networking tools like Wireshark and Linux commands can be used to analyze and troubleshoot network traffic.